T06

PROGRAMACIÓN ORIENTADA A OBJETOS: SOBRECARGA DEL OPERADOR (I)

INTRODUCCIÓN

```
>>> # Suma dos números
>>> 1 + 2
3

>>> # Concatenamos dos cadenas de texto
>>> 'Py' + 'thon'
'Python'


>>> # Multiplicamos
>>> 3 * 2
6

>>> # Repetimos la cadena de texto
>>> 'Python' * 3
'PythonPythonPython'
```

Ya hemos utilizado el operador + o * en un objeto *str* en Python, y su comportamiento es diferente en comparación con los objetos *int* o *float*.

EL MODELO DE DATOS DE PYTHON

- Consideremos una clase que representa un pedido online, la cual tiene un carrito (una lista) y un cliente (una cadena o instancia de otra clase que representa un cliente).

- En tal caso, es bastante natural querer obtener la longitud de la lista del carrito. Alguien nuevo en Python podría decidir implementar un método llamado obtener_num_objetos_carrito() en su clase para hacer esto. Pero podemos configurar el *len()* incorporado de tal manera que devuelva la longitud de la lista del carrito cuando se le proporcione nuestro objeto.

- También es posible que queramos agregar algo al carrito. Nuevamente, alguien nuevo en Python pensaría en implementar un método llamado anyanir_al_carrito() que recibe un artículo y lo agrega a la lista del carrito. Pero podemos configurar el operador + de tal manera que agregue un nuevo artículo al carrito.

EL MODELO DE DATOS DE PYTHON

- Python hace todo esto usando métodos especiales. Estos métodos especiales tienen una convención de nomenclatura, donde el nombre comienza con dos guiones bajos, seguido de un identificador y termina con otro par de guiones bajos.

- Esencialmente, cada función u operador integrado tiene un método especial correspondiente. Por ejemplo, está *__len__()*, correspondiente a *len()*, y *__add__()*, correspondiente al operador *+*.

- De forma predeterminada, la mayoría de los operadores y funciones integradas no funcionarán con objetos de nuestras clases. Para eso, debemos agregar los métodos especiales correspondientes en la definición de nuestra clase para que nuestro objeto sea compatible con funciones integradas y operadores.

- Cuando hacemos esto, el comportamiento de la función u operador asociado a ella cambia según lo definido en el método.

ASPECTOS INTERNOS DE OPERACIONES COMO LEN() Y []

- Cada clase en Python define su propio comportamiento para funciones y métodos integrados. Cuando pasamos una instancia de alguna clase a una función incorporada o usamos un operador en la instancia, en realidad es equivalente a llamar a un método especial.

- Si hay una función incorporada, *func()*, y el método especial correspondiente para la función es *__func__()*, Python interpreta una llamada a la función como *obj.__func__()*, donde *obj* es el objeto.

- En el caso de los operadores, si tiene un operador *opr* y el método especial correspondiente es *__opr__()*, Python interpreta *obj1 <opr> obj2* como *obj1.__opr__(obj2)*.

- Por ejemplo, cuando llamamos a *len()* en un objeto, Python maneja la llamada como *obj.__len__()*.

- Por lo tanto, cuando definimos estos métodos especiales en nuestra propia clase, anulamos el comportamiento de la función u operador asociado con ellos porque Python está llamando a nuestro método.

Todas las funciones incorporadas en Python: https://docs.python.org/es/3/library/functions.html

SOBRECARGA DEL OPERADOR

```
>>> a = 'Aprende Python'
>>> b = ['Aprende', 'Python']
>>> len(a)
14
>>> a.__len__()
14
>>> b[1]
'Python'
>>> b.__getitem__(1)
'Python'
```

Cuando usamos la función o su método especial correspondiente, obtenemos el mismo resultado.

SOBRECARGA DEL OPERADOR

```
>>> dir(a)
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__dir__',
 ...,
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 ...,
 'swapcase',
 'title',
 'translate',
 'upper',
 'zfill']
```

Cuando obtenemos la lista de atributos y métodos de un objeto *str* usando *dir()*, vemos estos métodos especiales en la lista, además de los métodos habituales disponibles en los objetos *str*

LONGITUD DE OBJETOS CON LEN()

```
>>> class Pedido:
...     def __init__(self, carrito, cliente):
...         self.carrito = list(carrito)
...         self.cliente = cliente
...
...     def __len__(self):
...         return len(self.carrito)
...

>>> pedido = Pedido([platano', manzana',\
          'mango'],'Miguel')

>>> len(pedido)
3
```

Para cambiar el comportamiento de *len()*, necesitamos definir el método especial *__len__()* en nuestra clase. Siempre que pase un objeto de nuestra clase a *len()*, se utilizará nuestra definición personalizada de *__len__()* para obtener el resultado.

LONGITUD DE OBJETOS CON LEN()

```
>>> class Pedido:
...     def __init__(self, carrito, cliente):
...         self.carrito = list(carrito)
...         self.cliente = cliente

>>> pedido = Pedido([platano', manzana',\
          'mango'],'Miguel')

>>> len(pedido)  # Calling len when no __len__
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'Pedido' has no len()
```

Cuando no tenemos el método *__len__()* definido, pero llamamos a *len()* en nuestro objeto, obtenemos un *TypeError*

LONGITUD DE OBJETOS CON LEN()

```
>>> class Pedido:
...     def __init__(self, carrito, cliente):
...         self.carrito = list(carrito)
...         self.cliente = cliente
...
...     def __len__(self):
...         return float(len(self.carrito))

>>> pedido = Pedido([platano', manzana',\
          'mango'],'Miguel')

>>> len(pedido)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'float' object cannot be interpreted as
an integer
```

Al sobrecargar *len()*, debemos tener en cuenta que Python requiere que la función devuelva un número entero. Si el método devolviera algo diferente a un número entero, obtendríamos un *TypeError*.

MÉTODO ABS()

```
>>> class Vector:
...     def __init__(self, x_comp, y_comp):
...         self.x_comp = x_comp
...         self.y_comp = y_comp
...
...     def __abs__(self):
...         return (self.x_comp ** 2 + \
                    self.y_comp ** 2) ** 0.5
...
>>> vector = Vector(3, 4)

>>> abs(vector)
5.0
```

En una clase que representa un vector en un espacio bidimensional, se puede usar *abs()* para obtener la longitud del vector.

EJERCICIO 6.1

REPR()

Obtiene la representación de cadena "parseable" de un objeto (se podría reconstruir)

STR()

Obtiene una representación de cadena fácil de usar del objeto que puede ser leída por un usuario normal

MÉTODO STR()

```
>>> class Vector:
...     def __init__(self, x_comp, y_comp):
...         self.x_comp = x_comp
...         self.y_comp = y_comp
...
...     def __str__(self):
...        return f'{self.x_comp}i{self.y_comp:+}j'
...
>>> vector = Vector(3, 4)

>>> str(vector)
'3i+4j'

>>> print(vector)
3i+4j
```

MÉTODO REPR()

```
>>> class Vector:
...     def __init__(self, x_comp, y_comp):
...         self.x_comp = x_comp
...         self.y_comp = y_comp
...
...     def __repr__(self):
...         return f'Vector({self.x_comp}, {self.y_comp})'
...

>>> vector = Vector(3, 4)

>>> repr(vector)
'Vector(3, 4)'

>>> vector  # En una sesión REPR, se usa el método repr()
'Vector(3, 4)'
```

En los casos en los que el método *__str__()* no está definido, Python usa el método *__repr__()* para imprimir el objeto.

OBJETOS VERDADEROS O FALSOS USANDO BOOL()

```
>>> class Pedido:
...     def __init__(self, carrito, cliente):
...         self.carrito = list(carrito)
...         self.cliente = cliente
...
...     def __bool__(self):
...         return len(self.cart) > 0
...
>>> pedido1 = Pedido([platano','manzana','mango'],'Miguel')
>>> pedido2 = Pedido([], 'Jorge')

>>> bool(pedido1)
True
>>> bool(pedido2)
False
```

Cuando el método especial *__bool__()* no se implementa en una clase, el valor devuelto por *__len__()* se utiliza como valor de verdad, donde un valor distinto de cero indica *True* y un valor cero indica *False*.

En caso de que ambos métodos no estén implementados, todas las instancias de la clase se consideran True.

EJERCICIO 6.2

USANDO + PARA AÑADIR ELEMENTOS A OBJETOS

```
>>> a = 'Aprende'

>>> a + 'Python'  # Crea un nuevo objeto
'AprendePython'

>>> a  # El valor no ha cambiado
'Aprende'

>>> a = a + 'Python'  # Crea un nuevo
        objeto y lo asigna a 'a'

>>> a
'AprendePython'
```

El uso del operador + en un objeto *str* devuelve una nueva instancia de *str*, manteniendo el valor de la instancia de llamada (*a*) sin modificar. Para cambiarlo, necesitamos asignar explícitamente la nueva instancia *a*.

ANTES DE SEGUIR, TENEMEOS QUE TENER CLAROS DOS CONCEPTOS…

PASO POR VALOR O REFERENCIA

- Dependiendo del tipo de dato que enviemos a la función / método, podemos diferenciar dos comportamientos:
    - **Paso por valor**: Se crea una copia local de la variable dentro de la función.
    - **Paso por referencia**: Se maneja directamente la variable, los cambios realizados dentro de la función le afectarán también fuera.
- En Python:
    - Los tipos simples se pasan por valor: *int*, *float*, *str*, *bool*...
    - Los tipos compuestos se pasan por referencia: *list*, *dict*, *set, object*...

PASO POR VALOR O REFERENCIA

```
>>> def doblar_valor(numero):
...     numero *= 2
...

>>> n = 10

>>> doblar_valor(n)

>>> n
10
```

PASO POR VALOR O REFERENCIA

```
>>> def doblar_valores(numeros):
...     for i,n in enumerate(numeros):
...         numeros[i] *= 2

>>> ns = [10,50,100]

>>> doblar_valores(ns)

>>> ns
[20, 100, 200]
```

PASO POR VALOR O REFERENCIA

```
>>> def doblar_valor(numero):
...     return numero * 2
...

>>> n = 10

>>> n = doblar_valor(n)

>>> n
20
```

Para modificar los tipos simples podemos devolverlos modificados y reasignarlos

PASO POR VALOR O REFERENCIA

```
>>> def doblar_valores(numeros):
...     for i,n in enumerate(numeros):
...         numeros[i] *= 2

>>> ns = [10,50,100]

>>> doblar_valores(ns[:])

>>> ns
[10, 50, 100]
```

Y en el caso de los tipos compuestos, podemos evitar la modificación enviando una copia

COPIANDO OBJETOS

```
>>> class Clase:
...     def __init__(self):
...         self.lista = ['hola']
...
...     def __str__(self):
...         return str(self.lista)

>>> o1 = Clase()

>>> o2 = o1

>>> o2.lista[0] = 'cambiado’

>>> f'{o1}, {o2}'
"['cambiado'], ['cambiado’]”

>>> f'{id(o1)}, {id(o2)}'
'4314663968, 4314663968'
```

o1 y o2 guardan una referencia al mismo objeto, con id 4350094480

COPIANDO OBJETOS

```
>>> o1 = Clase()

>>> o2 = Clase()

>>> o2.lista = o1.lista

>>> o2.lista[0] = 'cambiado'

>>> f'{o1}, {o2}'
"['cambiado'], ['cambiado']"

>>> f'{id(o1)}, {id(o2)}'
'4314081360, 4314082000'

>>> f'{id(o1.lista)}, {id(o2.lista)}'
'4314801472, 4314801472'
```

Ahora los objetos son diferentes, pero ambos guardan una referencia a la misma lista, con id 4335325312

COPIANDO OBJETOS

```
>>> o1 = Clase()

>>> o2 = Clase()

>>> o2.lista = o1.lista[:]

>>> o2.lista[0] = 'cambiado'

>>> f'{o1}, {o2}'
"['hola'], ['cambiado']"

>>> f'{id(o1)}, {id(o2)}'
'4314548384, 4314548688'

>>> f'{id(o1.lista)}, {id(o2.lista)}'
'4314792256, 4314802624'
```

Ahora los objetos son diferentes, y las listas son diferentes

COPIANDO OBJETOS

```
>>> import copy

>>> o1 = Clase()

>>> o2 = copy.deepcopy(o1)

>>> o2.lista[0] = 'cambiado'

>>> f'{o1}, {o2}'
"['hola'], ['cambiado']"

>>> f'{id(o1)}, {id(o2)}'
'4314413488, 4314098112'

>>> f'{id(o1.lista)}, {id(o2.lista)}'
'4314801472, 4314800832'
```

Alternativamente, podemos usar la función *deepcopy()* del módulo *copy*, que crea una copia "profunda" del objeto proporcionado.

No confundir con *copy.copy()*, que crea una copia superficial, equivalente a lo visto hace 2 diapositivas

EJERCICIO 6.3